SECTION 05 · AUTOENCODERS AND SEMI-SUPERVISED LEARNING

Autoencoders and Semi-Supervised Fraud Detection

Train networks to rebuild their input, score fraud by how badly they fail, then add a few labels.

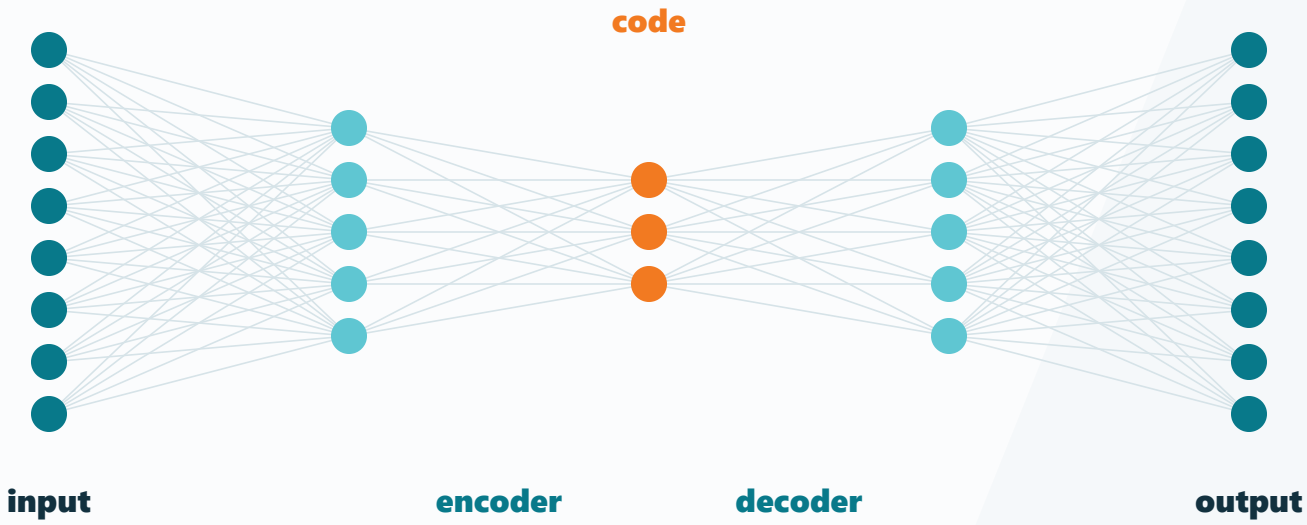
Keras 3 | PyTorch backend | Dropout | Denoising | SelfTrainingClassifier

KGP Talkie

Scan to subscribe

Laxmi Kant Tiwari · kgptalkie.com

What an Autoencoder Is



A network trained to reproduce its own input

The target is the input itself, so no labels are needed

The narrow middle
forces a summary

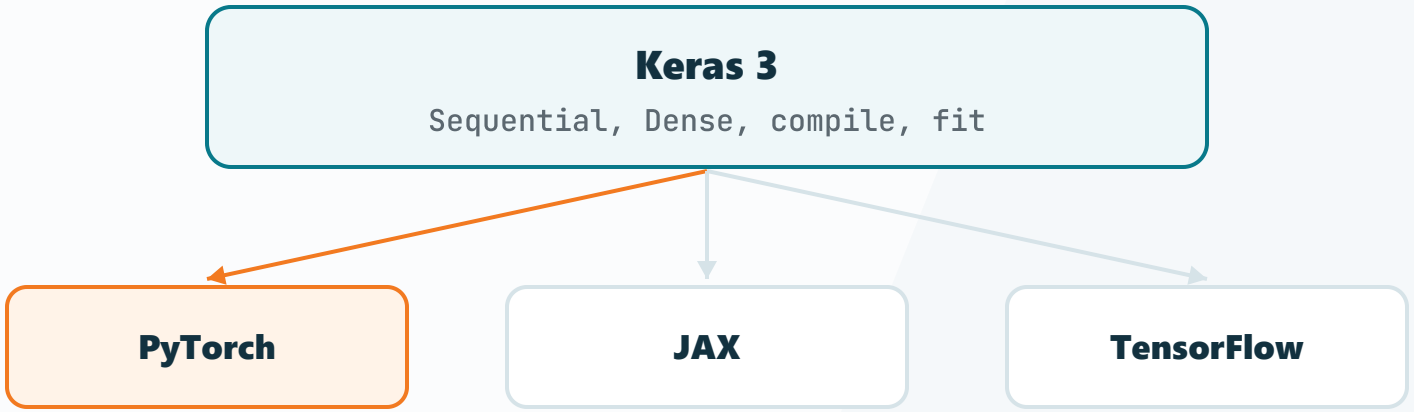
The Bottleneck as Compression



Room for common patterns only

normal transactions are rebuilt well; rare ones are rebuilt badly

Keras 3 on the PyTorch Backend



One front end, a choice of engines

set KERAS_BACKEND before the first import of keras; it cannot change later

Building and Training a Keras Model

`Sequential`

stack the layers:
29 → 10 → 29

→

`compile`

optimizer: Adam
loss: mean squared error

→

`fit`

inputs and targets
are the same data

→

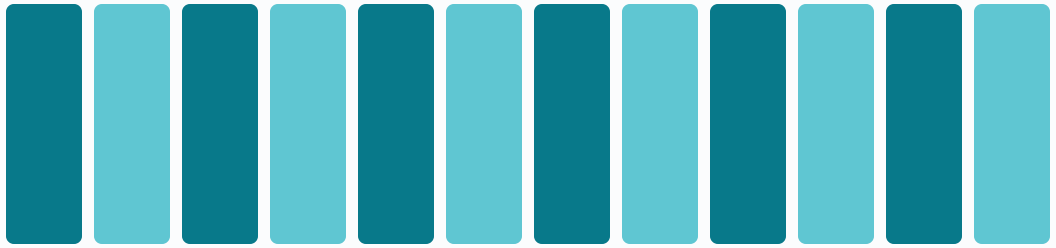
`predict`

rebuild every row,
then measure the error

A Dense layer with no activation is a matrix product plus a bias
so this first model can only learn linear combinations of the features

Batches and Epochs

the training rows, cut into batches



one weight update per batch; one pass over all batches is an epoch

The loss curve shows when to stop
it falls fast, then flattens

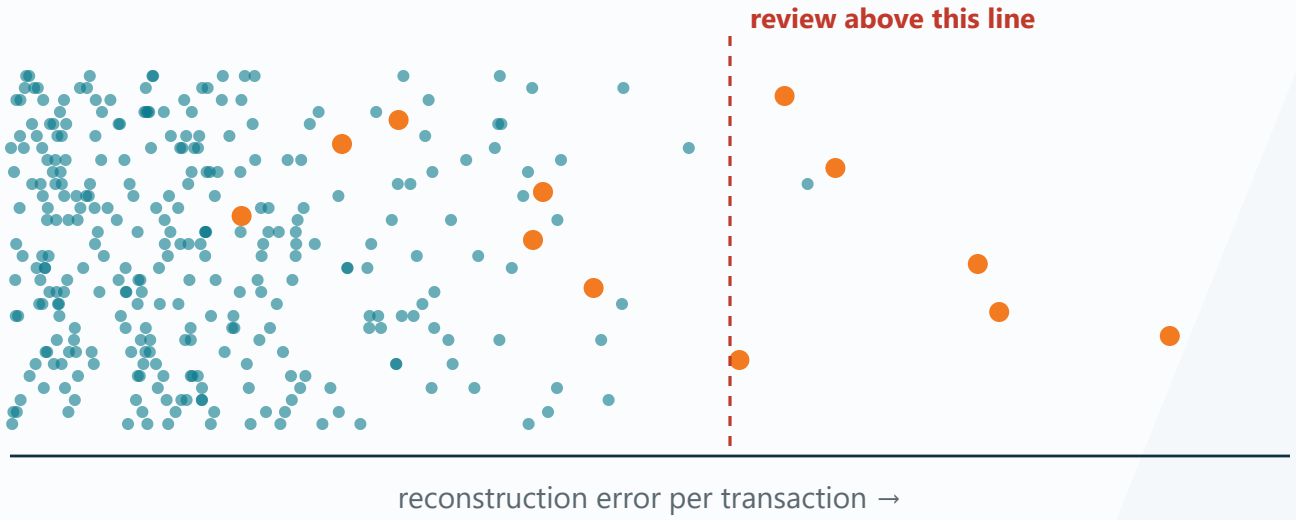
Batch size

small: many noisy updates, slow on a GPU
large: fewer, smoother updates, much faster

Epochs

too few: the model is still learning
many: it may start copying everything

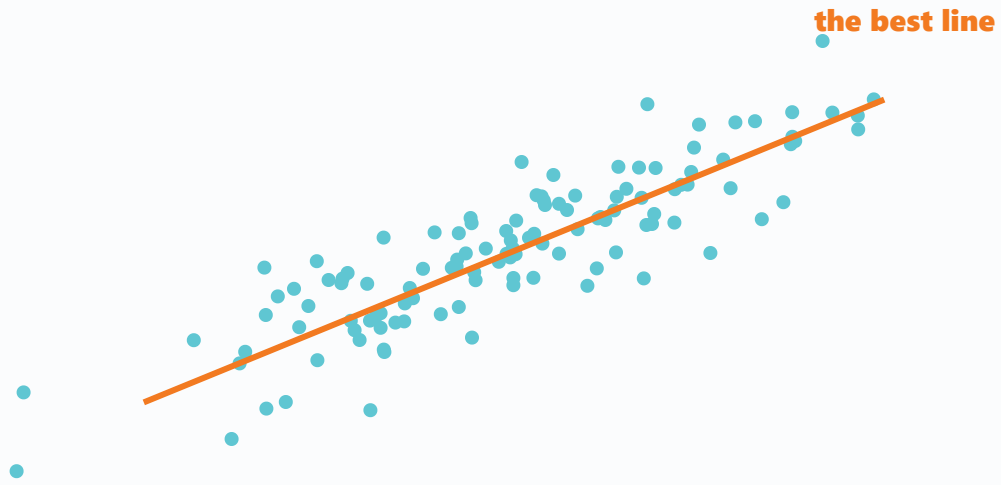
Reconstruction Error as a Fraud Score



The same score as before

sum of squared differences between a row and its rebuild, ranked from largest down

A Linear Autoencoder Is PCA



PCA and a linear autoencoder find the same flat space

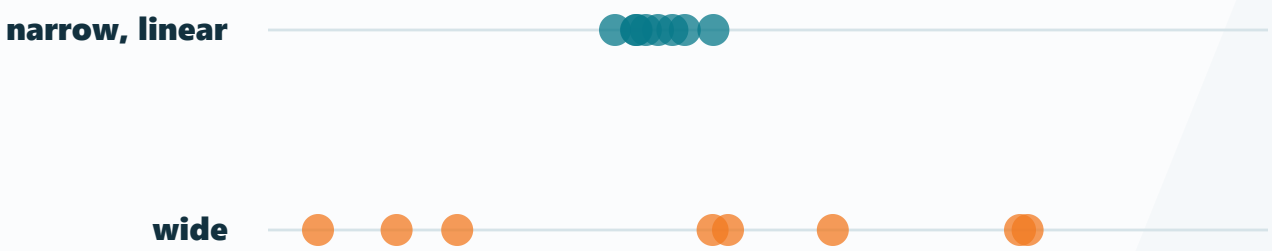
No activations, squared-error loss

The best a linear encoder and decoder can do is project onto the top components

Same space as PCA, but the 10 units need not line up with PCA's 10 axes

Expect the same fraud score as PCA
the autoencoder only pays off once it bends

Why Scores Change Between Runs



the score from eight training runs of the same model

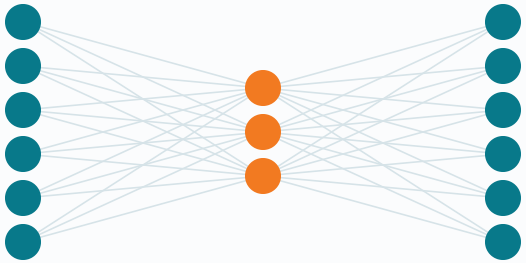
Why scores move

random starting weights
random batch order
very few frauds to rank

Trust a model that is steady
or average several runs
before comparing

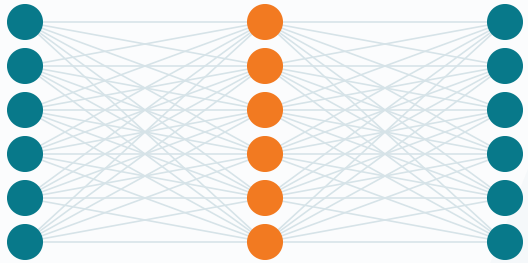
How Wide the Bottleneck Is

undercomplete



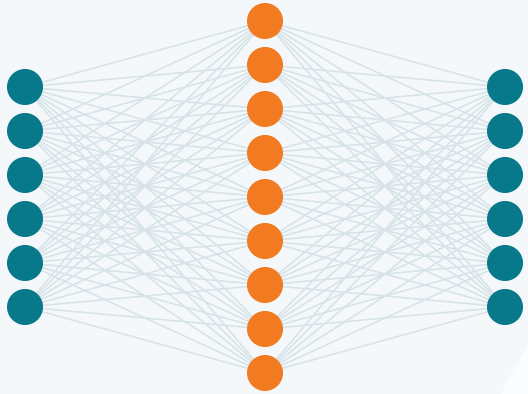
must summarise

complete



can keep everything

overcomplete

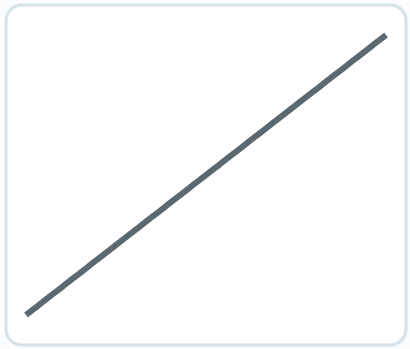


room to copy

A code as wide as the input is able to pass every row straight through

the closer it gets, the more frauds rebuild well; how far it gets varies from run to run

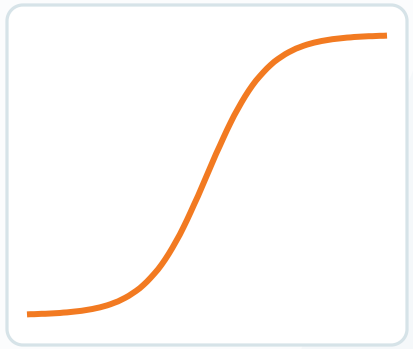
Activations and Depth



linear



ReLU



tanh

Activations let the model bend

curved patterns PCA cannot follow

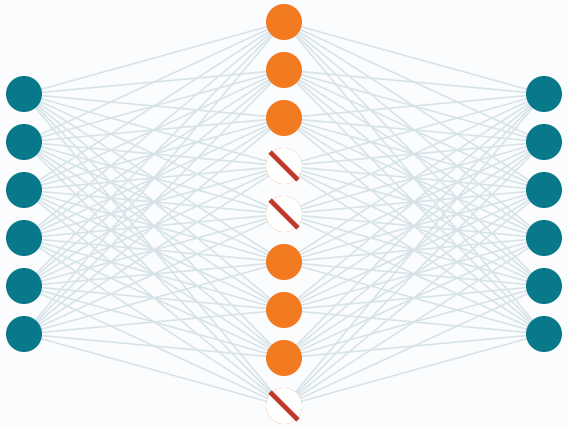
More layers bend more, but have more weights to fit in a few epochs

More power is not a better score

test each change, one at a time

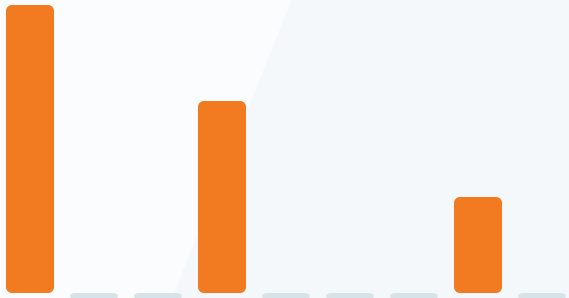
Dropout and Sparsity

Dropout



random units switched off each batch

Sparsity (L1 penalty)

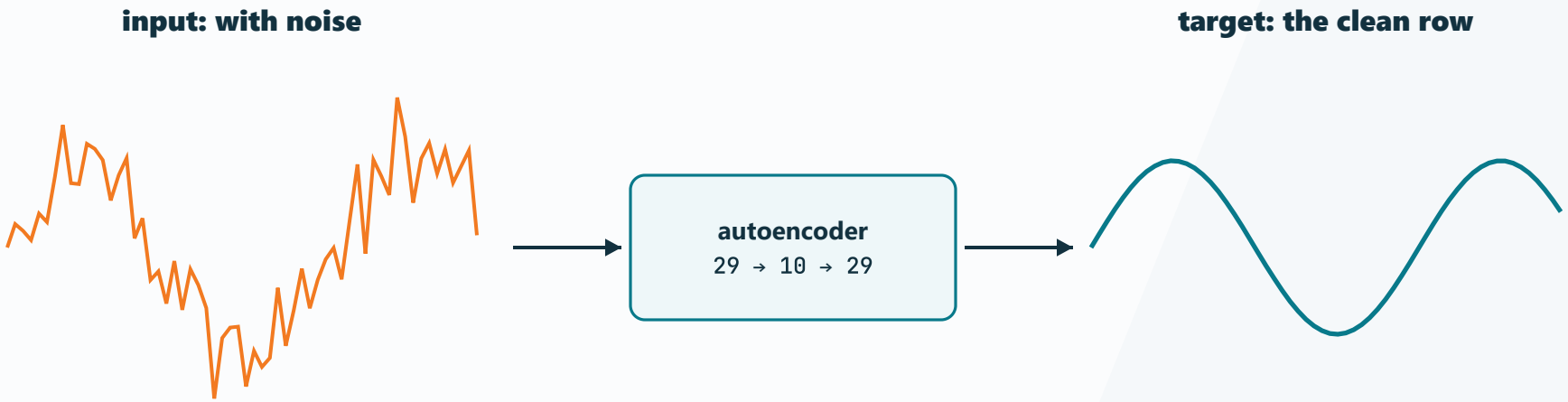


most code units stay near zero

Both stop copying

the wide layer has
to share the work
or use few units

Denoising Autoencoders



It cannot copy the input, because the input is wrong

it has to learn what a real transaction looks like to remove the noise

Semi-Supervised Learning

Unsupervised

no labels

score by how unusual

Semi-supervised

a few labels

plus many unlabelled rows

Supervised

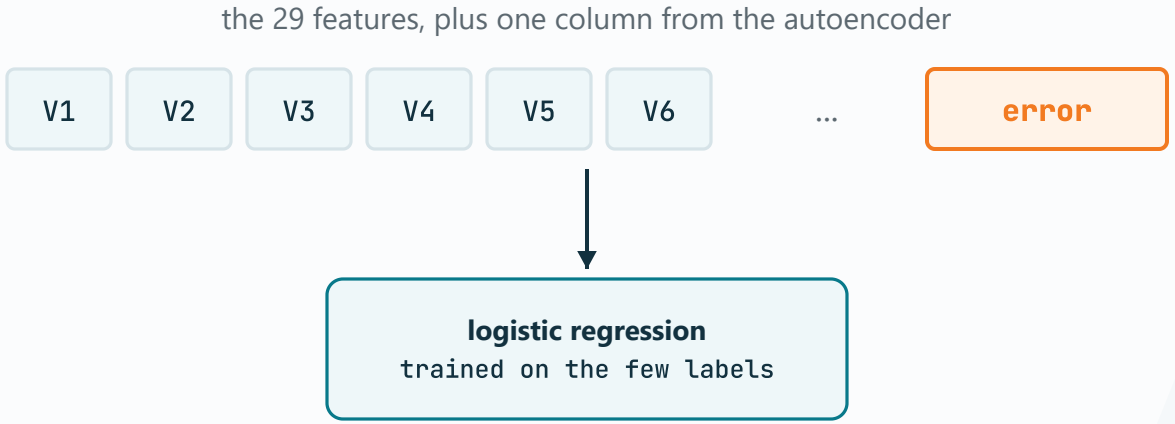
every row labelled

rarely affordable

Fraud teams check only a small share of transactions

a few confirmed cases are worth a lot; the rest of the data still helps

The Error as an Extra Feature

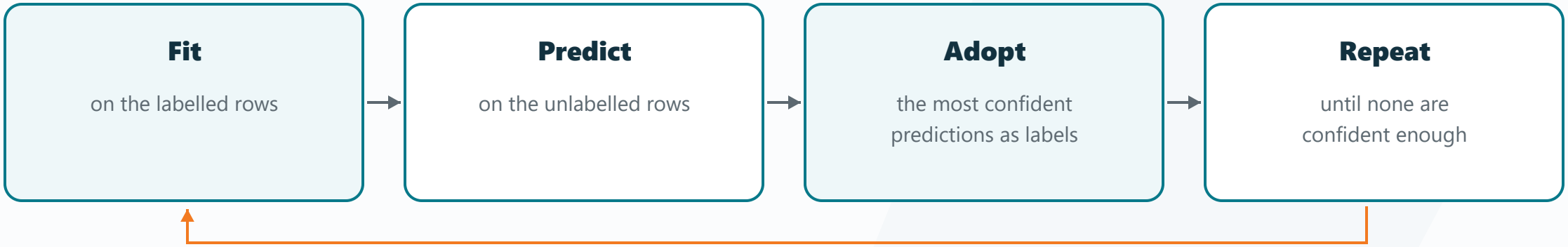


Unsupervised work feeds the supervised model

The autoencoder learned from every row; the classifier from a handful

It can help or not
check on the test set, every time

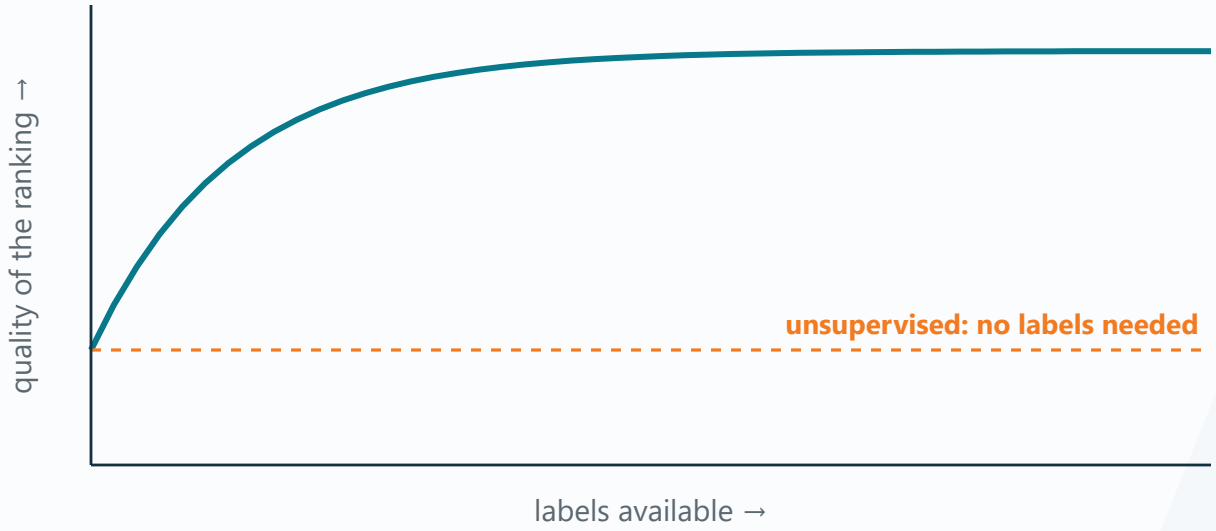
How Self-Training Works



Pseudo-labels: the model's own guesses become training data

a wrong guess gets reinforced, so the confidence threshold matters

What a Few Labels Buy

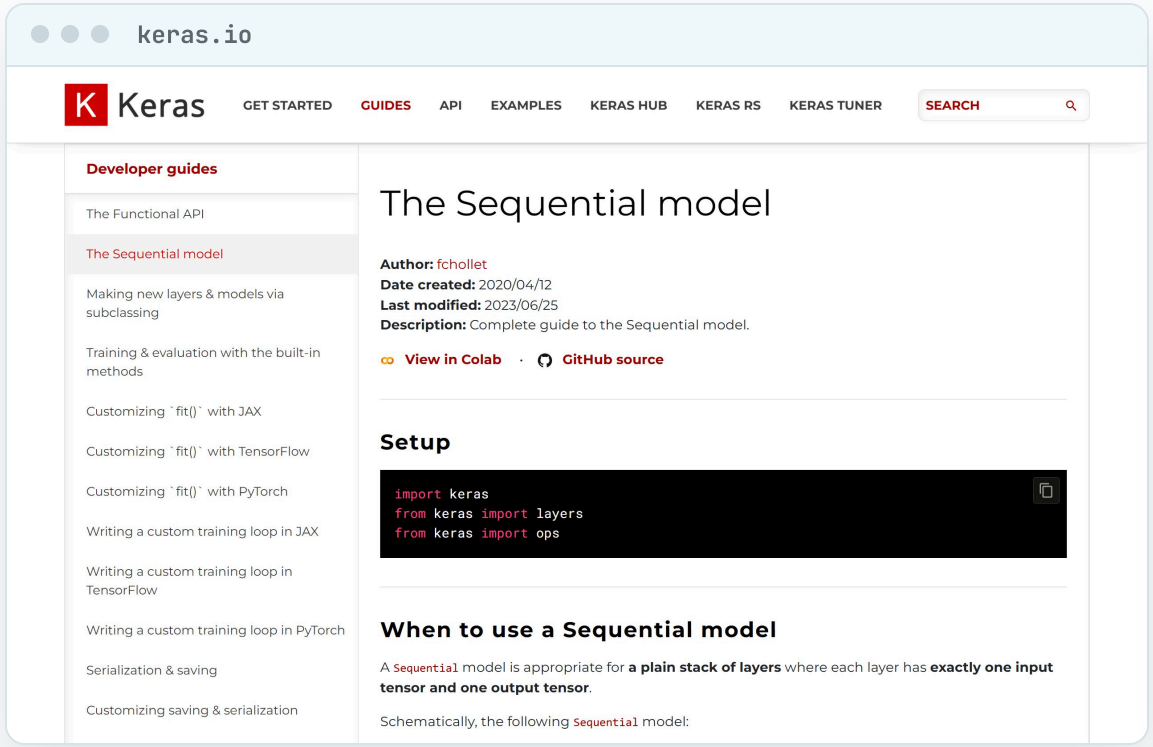


A first handful of labels buys the biggest jump

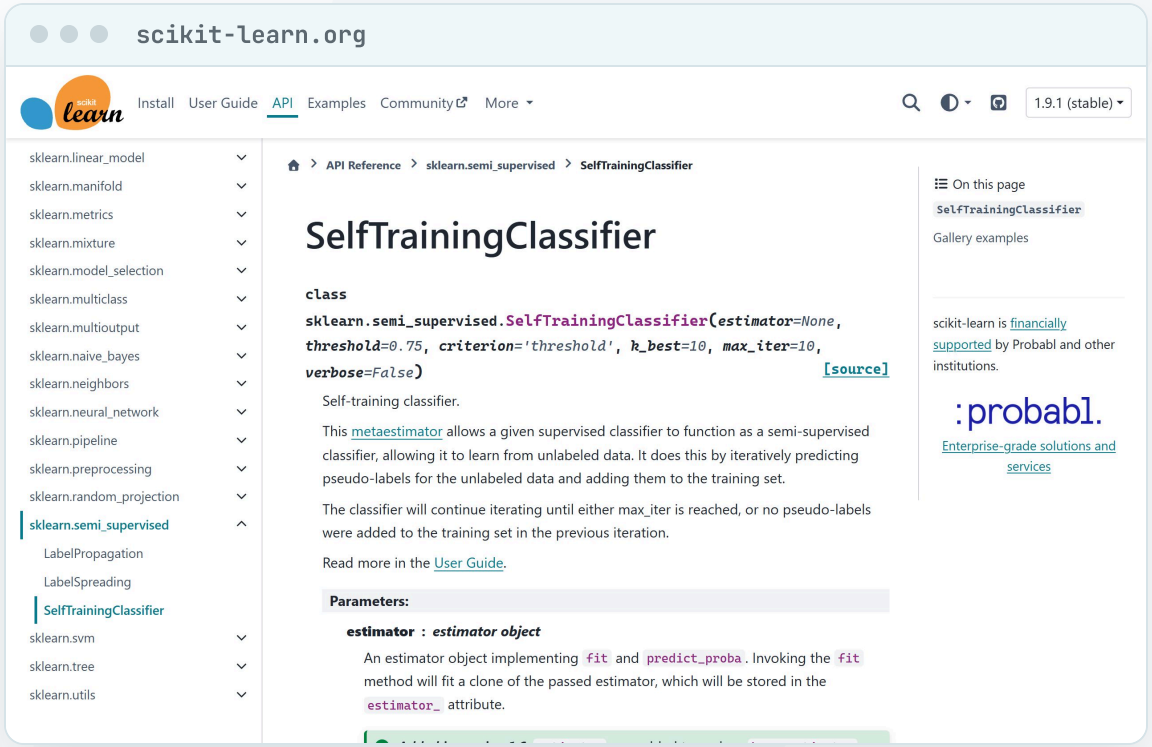
Each further label adds less

Start unsupervised, then label
the top of the anomaly list first:
that is where the frauds are

Keras and scikit-learn Self-Training Docs



Keras: the Sequential model guide



scikit-learn: SelfTrainingClassifier

Keras 3 runs the same code on PyTorch, JAX or TensorFlow.

Choosing an Approach

SITUATION	START WITH	WHY
No labels, tabular data	PCA reconstruction error	fast, steady, no tuning
Curved, complex patterns	an undercomplete autoencoder with activations	bends where PCA cannot
Wide autoencoder	dropout, an L1 penalty or noisy input	stops it copying the input
A few labels	a simple classifier, plus the error as a feature	even a handful of labels usually beats an unsupervised score
Many unlabelled rows	`SelfTrainingClassifier`	spreads labels, but check it on held-out data

SECTION 05 · COMPLETE

Thank You

Follow KGP Talkie for new lessons, course updates and live sessions.

Scan to follow

Instagram
@kgptalkie

WhatsApp Channel
KGP Talkie